

Practical No.	Laboratory Assignments
Cloud Computing	
1	Case study on Amazon EC2 and learn about Amazon EC2 web services.
2	Installation and configure Google App Engine.
3	Creating an Application in Salesforce.com using Apex programming Language.
4	Design and develop custom Application (Mini Project) using Sales force Cloud.
5	Mini-Project Setup your own cloud for Software as a Service (SaaS) over the existing LAN in your laboratory. In this assignment you have to write your own code for cloud controller using open-source technologies to implement with HDFS. Implement the basic operations may be like to divide the file in segments/blocks And upload/download file on/from cloud in encrypted form.

Practical No: 01

Title	Case study on Amazon EC2 and learn about Amazon EC2 web services.
Objectives	To learn Amazon EC2 web services; To study Amazon EC2 and learn about Amazon EC2 web services.
Hardware Requirements	Pentium IV with latest configuration
Software Requirements	Ubuntu 20.04

Theoretical Background

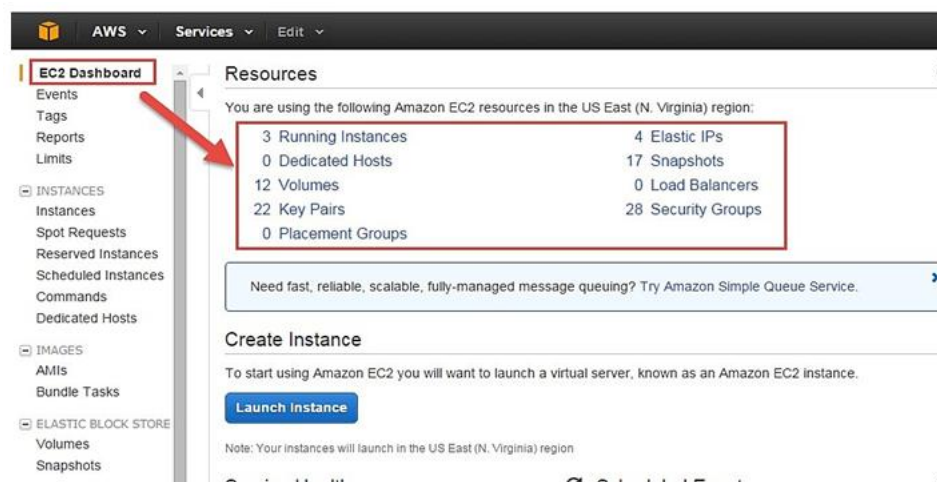
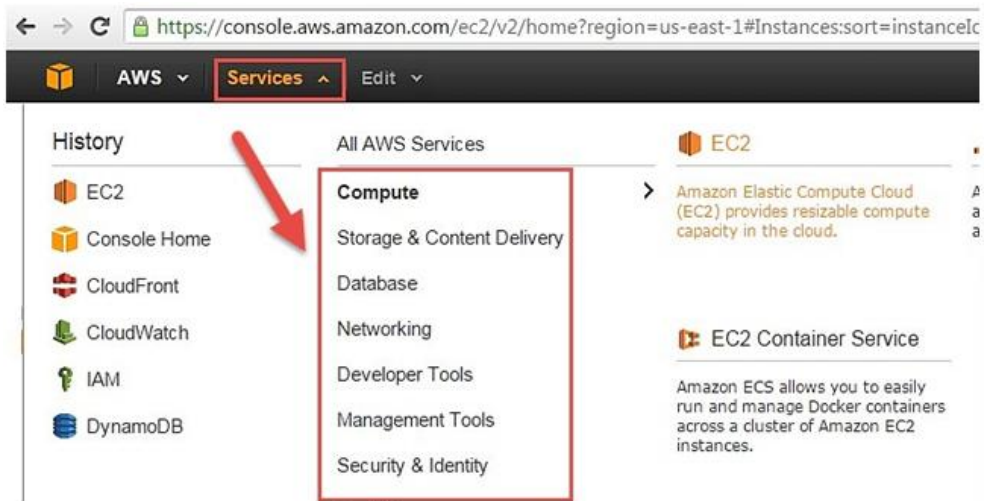
Amazon EC2, or **Elastic Compute Cloud**, represents the core compute layer of the AWS ecosystem. From an architectural perspective, an EC2 instance is a virtual server that abstracts physical hardware into scalable, on-demand compute resources.

The On-Demand Instance Model and CAPEX/OPEX: AWS utilizes an on-demand consumption model where subscribers provision virtual servers by the hour.

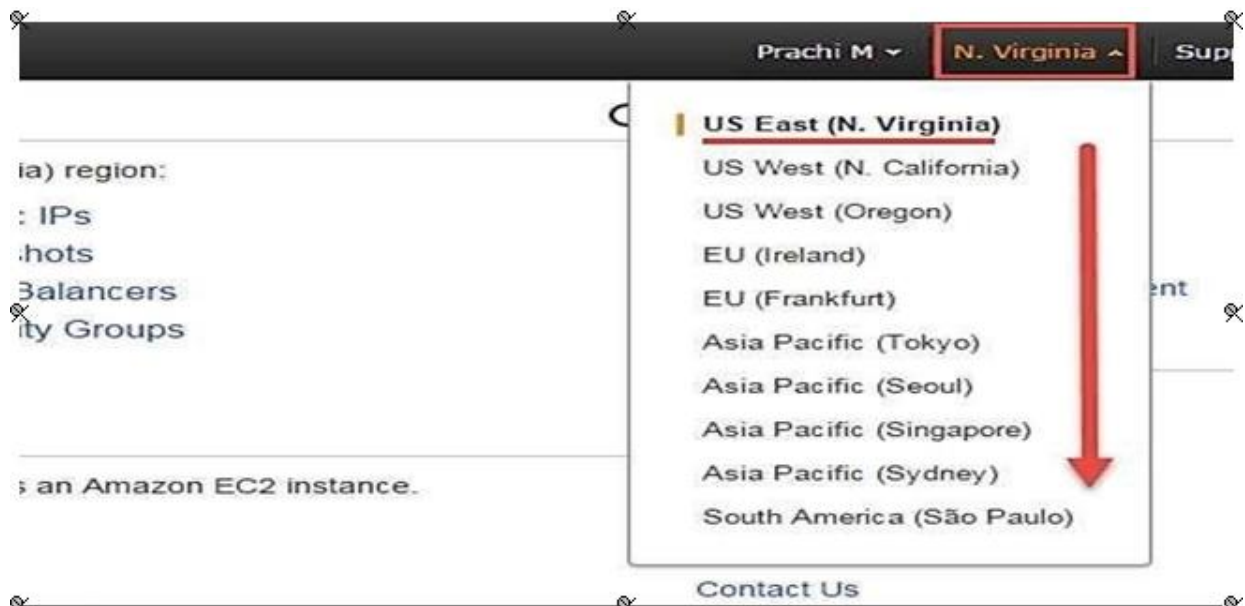
- **Operational Efficiency:** Users only pay for active compute time, allowing for the termination of resources when they are idle.
- **CAPEX to OPEX:** By renting infrastructure instead of purchasing physical hardware, organizations eliminate heavy Capital Expenditure (CAPEX) in favor of predictable Operating Expenditure (OPEX).
- **Scalability:** Architects can select specialized instance types optimized for different workloads (Compute, Memory, or Storage optimized) to meet specific business requirements.

Step-by-Step Procedure for Launching an Instance

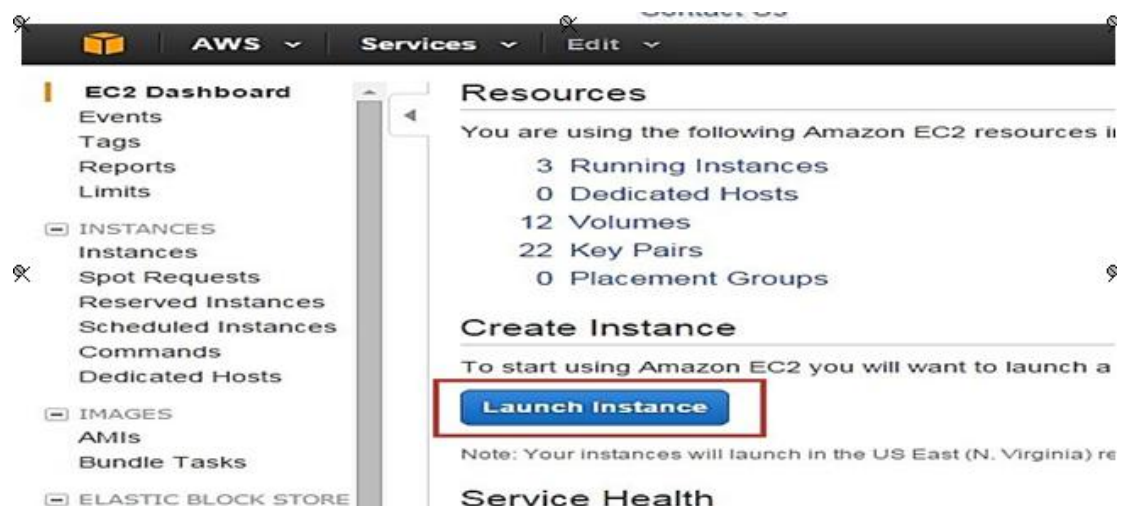
1. **Login and Service Selection:** Authenticate into the AWS Management Console. Navigate to the **Services** menu. Under the **Compute** category, select **EC2** to open the EC2 Dashboard, which provides a high-level view of your regional resources.



- Region Selection:** In the navigation bar (top right), select the target AWS Region. As per the laboratory standards (selecting from the 10 regions available in this environment), we will select **N. Virginia**.

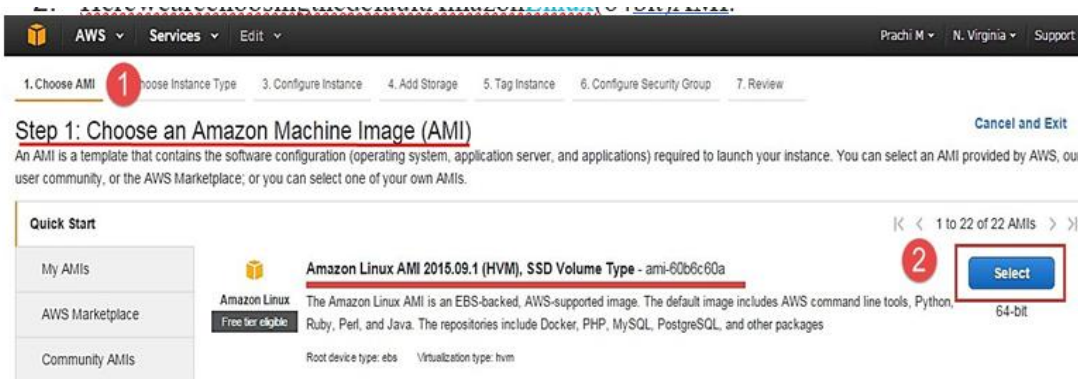


3. **Launch Command:** From the EC2 Dashboard, click the **Launch Instance** button to initiate the instance creation wizard.

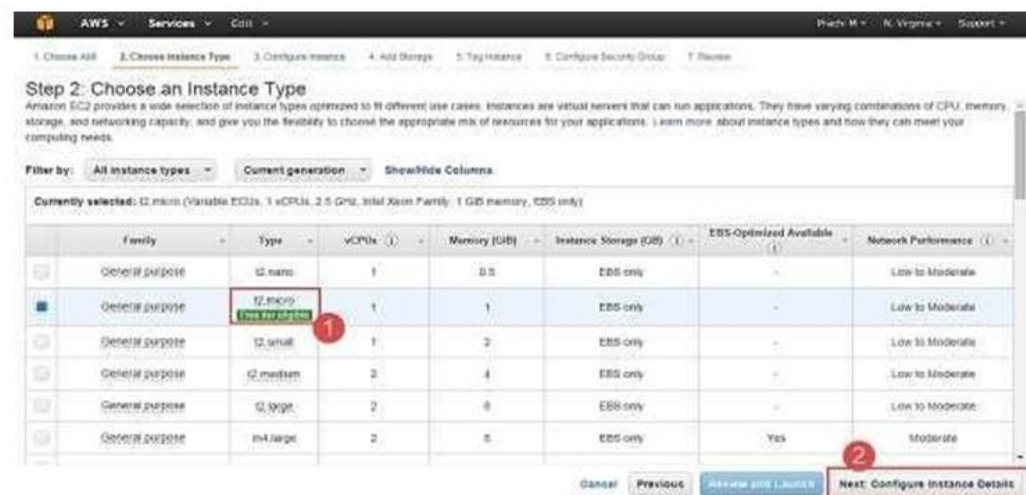


Instance Creation Wizard

1. **Choosing an AMI:** Select an **Amazon Machine Image (AMI)**. An AMI serves as the template containing the software configuration (operating system, application server, and applications) required to launch your instance.

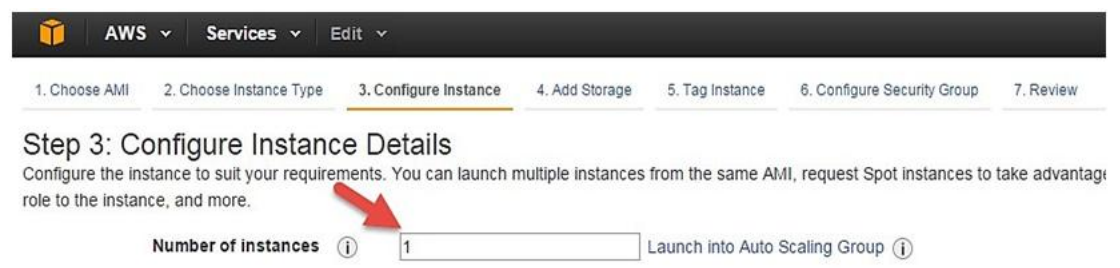


- Choosing Instance Type:** Select the hardware profile. We select **t2.micro**—a burstable performance instance providing 1 vCPU and 1 GB RAM. This is ideal for lab environments and qualifies for the AWS Free Tier. Highlighted parts in the below image (**t2.micro** and **“Next”**).



3. Configure Instance Details:

- Number of Instances:** Define the fleet size (e.g., 1). AWS allows provisioning up to 20 instances simultaneously.



- Purchasing Options:** Ensure "Request Spot Instances" remains unchecked to maintain an On-Demand billing cycle.

AWS Services Edit

1. Choose AMI 2. Choose Instance Type **3. Configure Instance** 4. Add Storage 5. Tag Instance 6. Configure Security Group 7. Review

Step 3: Configure Instance Details

Configure the instance to suit your requirements. You can launch multiple instances from the same AMI, request Spot instances to take advantage of role to the instance, and more.

Number of instances Launch into Auto Scaling Group

Purchasing option ☐ Request Spot instances

- Network/VPC:** Select the Virtual Private Cloud (VPC) where the instance will be isolated.

AWS Services Edit

1. Choose AMI 2. Choose Instance Type **3. Configure Instance** 4. Add Storage 5. Tag Instance 6. Configure Security Group 7. Review

Step 3: Configure Instance Details

Configure the instance to suit your requirements. You can launch multiple instances from the same AMI, request Spot instances to take advantage of role to the instance, and more.

Number of instances Launch into Auto Scaling Group

Purchasing option ☐ Request Spot instances

Network Create new VPC

Subnet Create new subnet

Auto-assign Public IP

IAM role Create new IAM role

4. Network and Subnet Configuration:

- Subnet:** Choose an existing Public Subnet within your VPC.

AWS Services Edit

1. Choose AMI 2. Choose Instance Type **3. Configure Instance** 4. Add Storage 5. Tag Instance 6. Configure Security Group 7. Review

Step 3: Configure Instance Details

Configure the instance to suit your requirements. You can launch multiple instances from the same AMI, request Spot instances to take advantage of the role to the instance, and more.

Number of instances Launch into Auto Scaling Group

Purchasing option ☐ Request Spot instances

Network Create new VPC

Subnet Create new subnet

Auto-assign Public IP

IAM role Create new IAM role

- Auto-assign Public IP:** Disable this feature for now. **Architect's Note:** In professional deployments, we often keep "Auto-assign Public IP" disabled to maintain control over connectivity. However, the instance will be unreachable via the internet until a static **Elastic IP (EIP)** is provisioned and associated with the instance in a later step to ensure persistence across restarts.

Step 3: Configure Instance Details

Configure the instance to suit your requirements. You can launch multiple instances from the same AMI, request Spot instances to take advantage of the Spot Instance market, request Reserved Instances to get a discount, or request Dedicated Hosts to launch on dedicated hardware. You can also launch the instance into an Amazon EC2 Auto Scaling group to help maintain the desired number of instances in your fleet.

Number of instances Launch into Auto Scaling Group

Purchasing option ☐ Request Spot instances

Network Create new VPC

Subnet Create new subnet
251 IP Addresses available

Auto-assign Public IP Use subnet setting (Disable) Enable Disable

IAM role Create new IAM role

Shutdown behavior

Step 3: Configure Instance Details

IAM role Create new IAM role

Shutdown behavior

Enable termination protection ☒ Protect against accidental termination

Monitoring ☐ Enable CloudWatch detailed monitoring
Additional charges apply.

Tenancy Shared - Run a shared hardware instance Shared - Run a shared hardware instance Dedicated - Run a Dedicated instance Dedicated host - Launch this instance on a Dedicated host

Step 3: Configure Instance Details
Configure the instance to suit your requirements. You can launch multiple instances from the same AMI, request Spot instances to take advantage of the lower pricing, assign an access management role to the instance, and more.

Number of instances ① [Launch into Auto Scaling Group](#) ①

Purchasing option ① ☒ On-Demand Instances

Network ① [Create new VPC](#)

Subnet ① [Create new subnet](#)
251 IP addresses available

Auto-assign Public IP ①

IAM role ① [Create new IAM role](#)

Shutdown behavior ①

Enable termination protection ① ☒ Protect against accidental termination

Monitoring ① ☐ Enable CloudWatch detailed monitoring
Additional charges apply

Tenancy ①

[Cancel](#) [Previous](#) [Review and Launch](#) [Next: Add Storage](#)

Launch Status



Your instances are now launching

The following instance launches have been initiated: i-4c2c3cff [Hide launch log](#)

Creating security groups	Successful (sg-62d7d21b)
Authorizing inbound rules	Successful
Initiating launches	Successful
Applying tags	Successful
Launch initiation complete	



Get notified of estimated charges

Create billing alerts to get an email notification when estimated charges on your AWS bill exceed an am

The screenshot shows the AWS Management Console interface for an EC2 instance. The instance is named 'Dev_Web Server 01' and is in a 'running' state. A red arrow points to the 'Private IP' field, which displays '192.168.2.167'.

Name	Instance ID	Instance Type	Availability Zone	Instance State	Status Checks	Alarm Status
Dev_Web Server 01	i-4c2c3cff	t2.micro	us-east-1b	running	Initializing	None

Instance: **i-4c2c3cff (Dev_Web Server 01)** Private IP: **192.168.2.167**

Description		Status Checks	Monitoring	Tags
Instance ID	i-4c2c3cff	Instance state	running	
Instance type	t2.micro	Private DNS	ip-192-168-2-167.ec2.internal	
Private IPs	192.168.2.167	Secondary private IPs		
VPC ID	vpc-d5194fb0	Subnet ID	subnet-43a3d0ea	
Network interfaces	eni0	Source/dest. check	True	
ClassicLink		EBS-optimized	False	
Public DNS	-	Public IP	-	
Elastic IP	-	Availability zone	us-east-1b	
Security groups	Web Server SG - view rules	Scheduled events	No scheduled events	
AMI ID	amazon-ami-hvm-2015.09.1.x86_64-gp2 (ami-60b6c62a)	Platform	-	
IAM role	-	Key pair name	Dev Key	
Owner	016611290429	Launch time	February 3, 2016 at 7:52:22 PM UTC+5:30 (less than one hour)	

Conclusion

This practical illustrates the workflow for provisioning and managing on-demand EC2 resources. By transitioning from physical servers to elastic virtual instances, organizations can optimize their cost structure and respond dynamically to fluctuating compute demands.

Practical No: 02

Title	Installation and configuration of Google App Engine.
Objectives	To learn the basics of Google App Engine; To install and configure Google App Engine.
Hardware Requirements	Pentium IV with latest configuration
Software Requirements	Ubuntu 20.04, Web Application (Google App Engine)

Theory of Google App Engine (GAE)

Google App Engine (GAE) is a fully managed, serverless platform designed for hosting web applications. It abstracts the underlying infrastructure, allowing developers to focus strictly on code while Google handles the operational overhead.

Scalability and Resource Allocation: GAE is architected for high-concurrency environments. It utilizes **Automatic Scaling**, meaning the platform monitors incoming traffic and dynamically allocates compute resources (spinning up or down instances) to ensure performance remains consistent without manual intervention.

Integrated Service Model: GAE represents a comprehensive cloud platform that bridges multiple service layers:

- **Infrastructure as a Service (IaaS):** Provides the fundamental compute and storage nodes.
- **Platform as a Service (PaaS):** Provides the runtime environments and development frameworks.
- **Software as a Service (SaaS):** Delivers integrated software management tools for the developer.

Features and Advantages

- **Standard Features:** Covered by the Google Service Level Agreement (SLA), these features include:
 - **Persistent Storage:** Data storage, retrieval, and full-text search.
 - **Managed Databases:** Cloud SQL, NoSQL Datastore, and Blobstore.

- **Caching:** Shared Memcache and Dedicated Memcache for low-latency data access.
 - **Computation:** Integrated process management and image processing modules.
- **Security Infrastructure:** Google's internet infrastructure is among the most secure globally. Code and data are encrypted and stored on hardened servers, benefiting from Google's rigorous global security and privacy policies.
- **Performance and Reliability:** By leveraging Google's global network of hundreds of servers, applications achieve high availability and the same reliability benchmarks as Google's core products.
- **Cost Savings:** The "Zero-ops" nature of GAE means businesses do not need to employ dedicated server engineers for maintenance, allowing reallocation of budget to core business development.

Platform Independence

GAE promotes flexibility by minimizing platform lock-in. Developers can migrate data to other environments with minimal friction, ensuring that the business is not strictly dependent on a single provider's ecosystem.

Conclusion

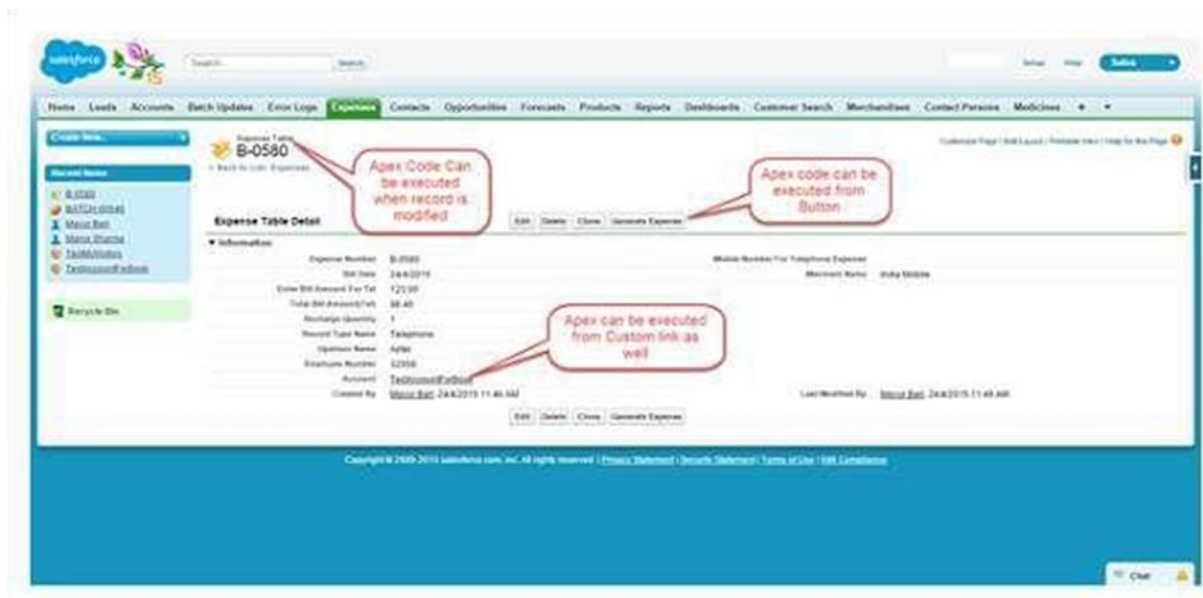
The successful configuration of Google App Engine provides a robust, scalable, and secure environment. It empowers developers to deploy high-performance applications with minimal infrastructure management.

Practical No: 03

Title	Creating an Application in Salesforce.com using Apex programming Language
Objectives	To learn Salesforce cloud administration; To create an application in Salesforce.com using Apex.
Hardware Requirements	Pentium IV with latest configuration
Software Requirements	Ubuntu 20.04, Web Application (Salesforce.com)

Introduction to Apex

Apex is a proprietary, strongly typed, object-oriented language developed by Salesforce.com. It is designed to execute flow and transaction control statements on the Force.com platform server in conjunction with the API.



Technical Specifications:

- **Availability:** Apex is included in the **Enterprise, Performance, Unlimited, and Developer** editions of Salesforce.
- **Java-like Syntax:** Uses familiar syntax for variable declarations and loop structures.

- **Data Integration:** Highly optimized for executing multiple SOQL queries and DML statements within a single transaction.

Operational Features

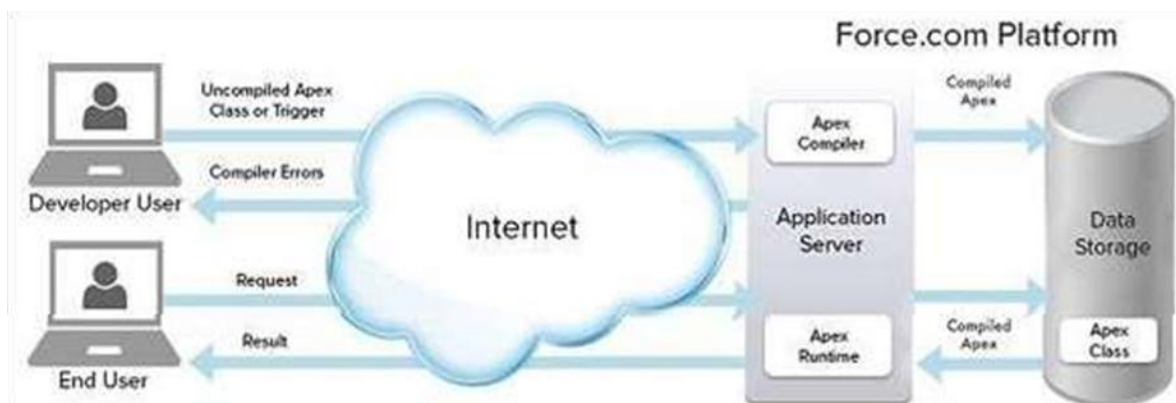
I recommend choosing Apex when standard "Point-and-Click" (declarative) configuration cannot meet complex requirements. Key use cases include:

- Developing custom email services for automated blasts.
- Implementing multi-object validation logic that exceeds standard validation rules.
- Executing complex transaction logic (e.g., using specific Database methods for partial updates).
- Managing triggers to respond to specific database events.

Working Structure and Flow

Apex execution follows a unique metadata-driven lifecycle:

1. **Developer Action:** When code is saved, the server compiles it into instructions for the Apex runtime interpreter. These are stored as **Metadata**.
2. **End User Action:** When an event (like a button click) triggers the code, the server fetches the instructions from metadata and executes them through the interpreter.



Syntax and Code Components

- **Variable Declaration:** Apex requires explicit data typing for every variable (e.g., `List<Account> lstAcc`).

```
// Declare a list of Account records
List<Account> lstAcc = new List<Account>();
```

```
// Add a new Account record
Account acc = new Account();
```

```
acc.Name = 'ABC Company';
```

```
// Add to list  
lstAcc.add(acc);
```

Explanation:

List<Account> → list that stores Account objects
Account → Salesforce object (like a table)

- **SOQL Queries:** Salesforce Object Query Language is used to retrieve data from the database.

```
// Fetch records from Account object  
List<Account> lstAcc = [SELECT Id, Name FROM Account];
```

```
// Display records  
for (Account acc : lstAcc) {  
    System.debug('Account Name: ' + acc.Name);  
}
```

Explanation:

SELECT Id, Name FROM Account → fetch data
Stored in lstAcc

- **Loop Statements:** Used to iterate through record collections retrieved from queries.

```
// Loop through Account records  
List<Account> lstAcc = [SELECT Id, Name FROM Account];
```

```
for (Account acc : lstAcc) {  
    System.debug('Processing Account: ' + acc.Name);  
}
```

Explanation:

Loop runs for each record
Used to process multiple records

- **Flow Control (If Statements):** Conditional logic used to prevent errors (e.g., checking if a list is empty before processing).

```
List<Account> lstAcc = [SELECT Id, Name FROM Account];
```



```
// Check if list is not empty
if (lstAcc.size() > 0) {
    for (Account acc : lstAcc) {
        System.debug('Account: ' + acc.Name);
    }
} else {
    System.debug('No records found');
}
```

Explanation:

Prevents errors when list is empty
Adds decision-making logic

- **DML Statements:** Data Manipulation Language (Insert, Update, Upsert, Delete) used to modify persistent records.

```
// Create new Account
Account acc = new Account(Name = 'XYZ Company');
// Insert record
insert acc;
// Update record
acc.Name = 'XYZ Company Updated';
update acc;

// Delete record
delete acc;
```

Explanation:

insert → add record
update → modify record
delete → remove record

Conclusion

Through Apex, developers can extend the Salesforce platform beyond standard UI capabilities, building data-intensive applications with complex server-side logic.

Reference: https://www.tutorialspoint.com/apex/apex_overview.html

Practical No: 04

Title	Design and develop custom Application (Mini Project) using Salesforce Cloud.
Objectives	To learn Salesforce cloud administration; To install and configure administrative features.
Hardware Requirements	Pentium IV with latest configuration
Software Requirements	Ubuntu 20.04, Web Application (Salesforce.com)

Theory of Salesforce Ecosystem

- **Salesforce.com:** A global leader in cloud-based CRM and enterprise SaaS solutions.
- **Lightning Platform (Force.com):** The underlying PaaS that supports custom application development using Apex and Visualforce.
- **Community Cloud:** A collaborative platform for building portals for customers or partners.
- **Sales Cloud:** A CRM platform for managing sales cycles. It is available in five tiers: **Group, Professional, Enterprise, Unlimited, and Performance.**

Implementation Guide: Creating a Custom App

To create a custom application in **Salesforce Classic**, follow this administrative path:

1. **Navigate to Setup:** Use the Quick Find box to search for and select **Apps**.
2. **New Application:** Click the **New** button.
3. **App Type:** Choose **Custom App** (unless a specific Console UI is required).
4. **Labeling:** Provide a Name and Description (Label max: 40 characters).
5. **Branding:** Select a custom logo to define the app's identity.
6. **Tab Selection:** Add the specific Objects and Tabs relevant to the application's purpose.
7. **Default Landing:** Specify which tab the user sees first upon login.

8. **Profile Assignment:** Determine which user profiles can access the app and set the default application for those profiles.

Comparative Analysis: Custom vs. Console

Feature	Custom Application	Console Application
Primary Goal	Grouping tabs/objects for specific business processes.	Enhancing user productivity and multitasking.
User Interface	Standard Salesforce Classic navigation.	Single-screen, tabbed interface for fast switching.
Best Use Case	General sales, marketing, or custom business modules.	High-volume environments like Customer Support or Telesales.
Navigation	Individual page refreshes.	Multiple records open as sub-tabs under a primary record.

Conclusion

By designing a custom application, administrators can provide users with a streamlined, branded interface that consolidates only the tools necessary for their specific organizational role.

Practical No: 05

Title	Mini-Project
	Setup your own cloud for Software as a Service (SaaS) over the existing LAN in your laboratory. In this assignment you have to write your own code for cloud controller using open-source technologies to implement with HDFS. Implement the basic operations may be like to divide the file in segments/blocks and upload/ download file on/from cloud in encrypted form.
Objectives	To set up a private cloud for SaaS over existing LAN; To implement basic operations like file segmentation and encryption.
Hardware Requirements	Pentium IV with latest configuration
Software Requirements	Ubuntu 20.04, VMware ESXi cloud

Theory and Hardware Requirements

VMware ESXi 6.7 is a **Type-1 Bare-Metal Hypervisor** that installs directly onto physical hardware. To support a private cloud environment, the host must meet these specifications:

- **CPU:** Minimum two cores, 64-bit x86 support. The **NX/XD bit** must be enabled in the BIOS.
- **Virtualization Support:** Intel VT-x or AMD RVI must be enabled for 64-bit VM support.
- **Memory:** 4 GB physical RAM minimum (8 GB recommended for SaaS delivery).
- **Networking:** Gigabit Ethernet controllers.
- **Storage:** Local SCSI, SATA, or RAID LUNs for virtual machine disk files.

ESXi Installation and Configuration Workflow

Steps for VMware ESXi Installation

To set up a VMware ESXi server, follow the steps below:

Step 1: Verify Hardware Compatibility

Check your server model and hardware components. Make sure your server model appears on **VMware's Hardware Compatibility List (HCL)**.

Enable virtualization support (Intel VT-x or AMD-V) in BIOS/UEFI. Decide on storage — internal disk, RAID array, or SAN/NAS.

Step 2: Download the ESXi ISO

Create a VMware Customer Connect account or log in if you already have an account. After logging in, download the ESXi ISO file matching your hardware. Save this file to your workstation.

Step 3: Create a Bootable USB Installer

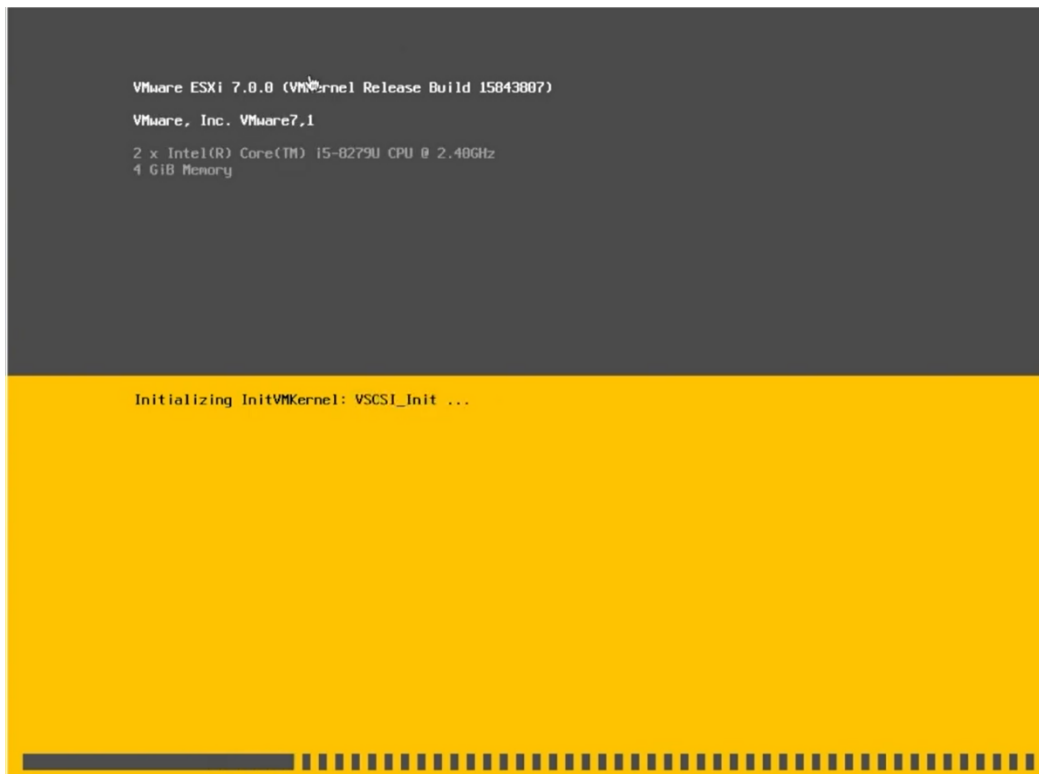
After downloading the ISO file, prepare a bootable USB drive using a tool like Rufus. This will allow you to install ESXi on your server.

1. First, insert a blank USB into your workstation.
2. Launch **Rufus** and choose your USB drive from the Device menu.
3. Under **Boot selection**, choose the ESXi ISO file.
4. Leave defaults as they are and click **Start**. Now, wait for Rufus to complete the process.

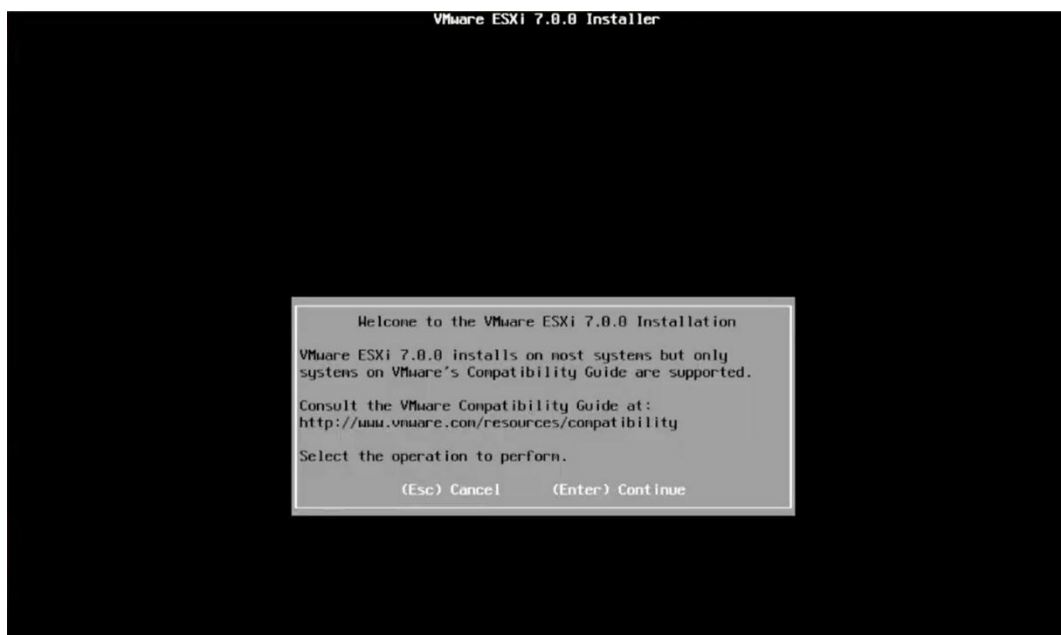
Step 4: Boot and Install ESXi

After creating a bootable drive, you can easily install VMware ESXi on your server by following these steps:

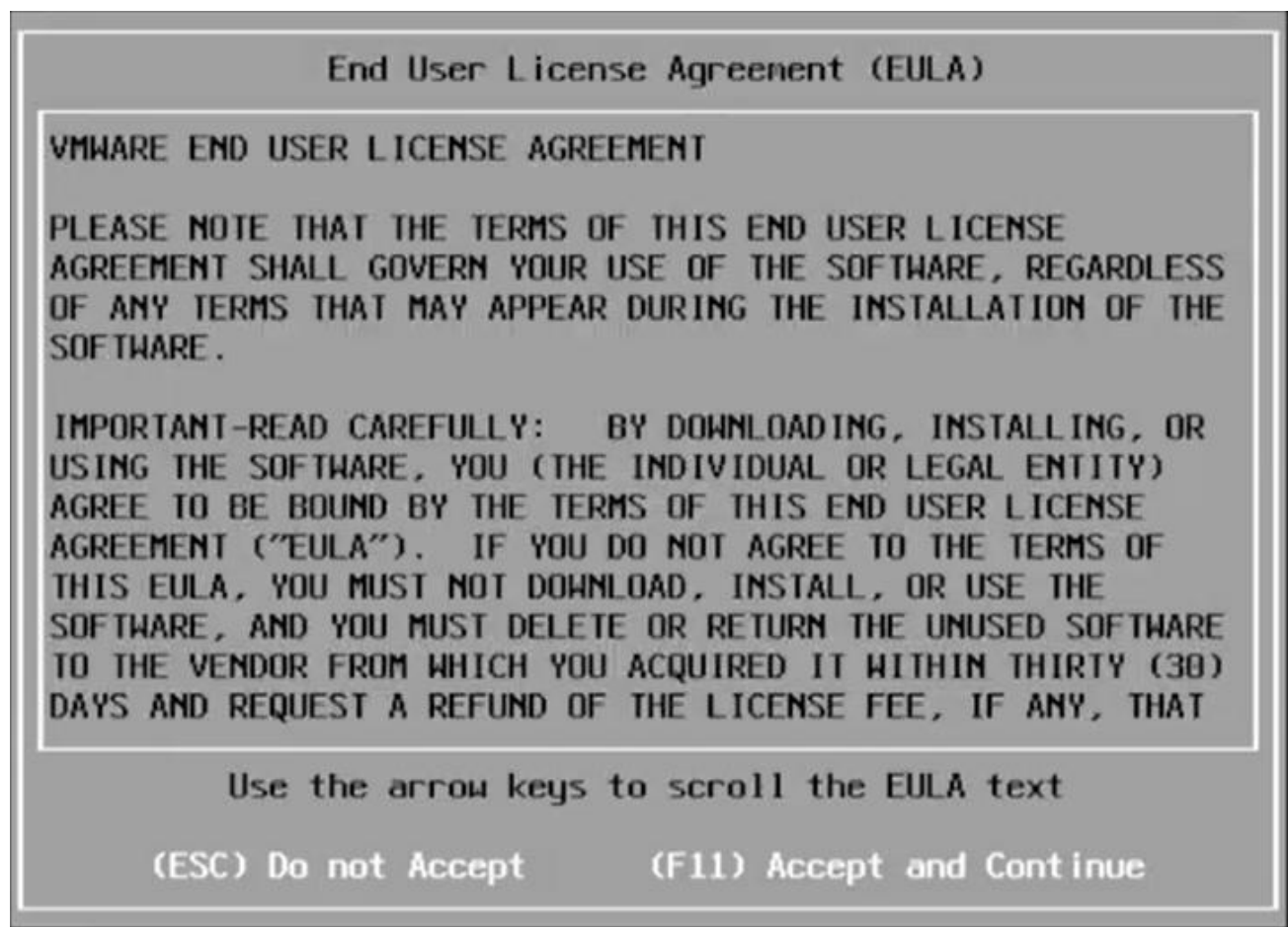
1. Plug the USB drive into your server.
2. Enter the BIOS/UEFI (often **accessed by pressing F2, Del, or F12**) and set USB as the primary boot device.
3. Boot into the ESXi installer.



4. Press **Enter** to continue the installation.



5. Accept the license agreement (**F11**).



6. Select the disk where ESXi will be installed (note: this will erase all existing data on the disk).



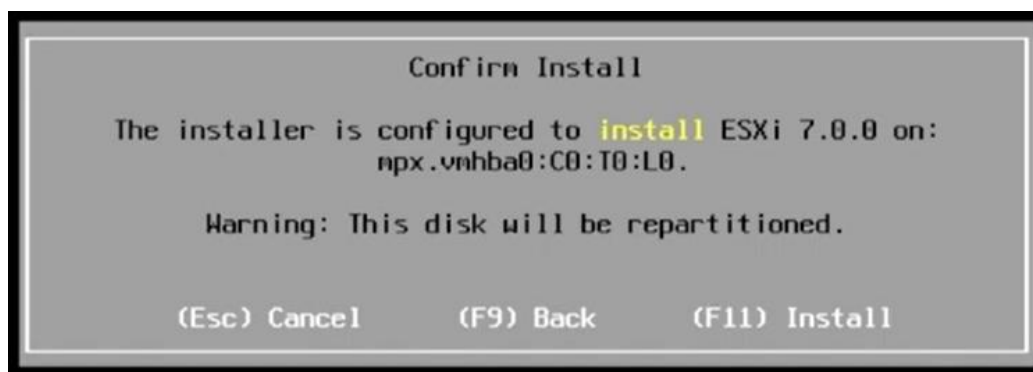
7. Select the Keyboard Layout from the list. Select **US Default**:



8. Set a **root** password.



9. Press **F11** to start installation.



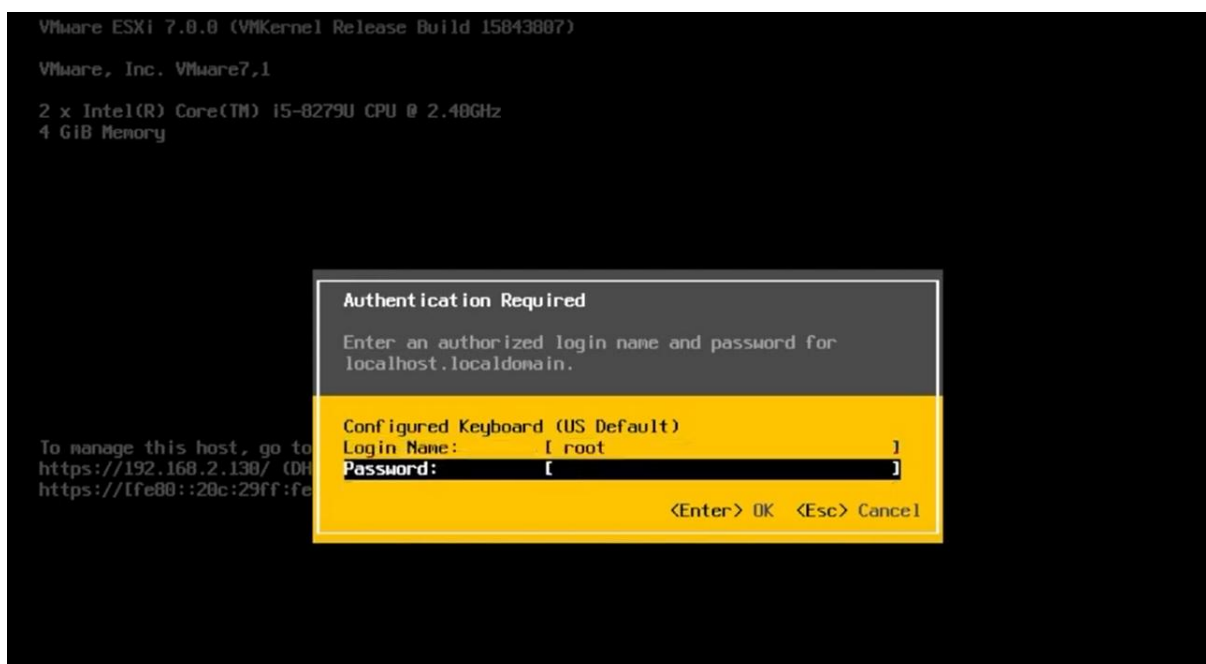
10. When done, remove the USB and reboot.



Step 5: Configure the Host (DCUI)

Once the server restarts, the **Direct Console User Interface (DCUI)** will be displayed.

1. Press **F2** and log in as **root**.



2. Select **Configure Management Network**.

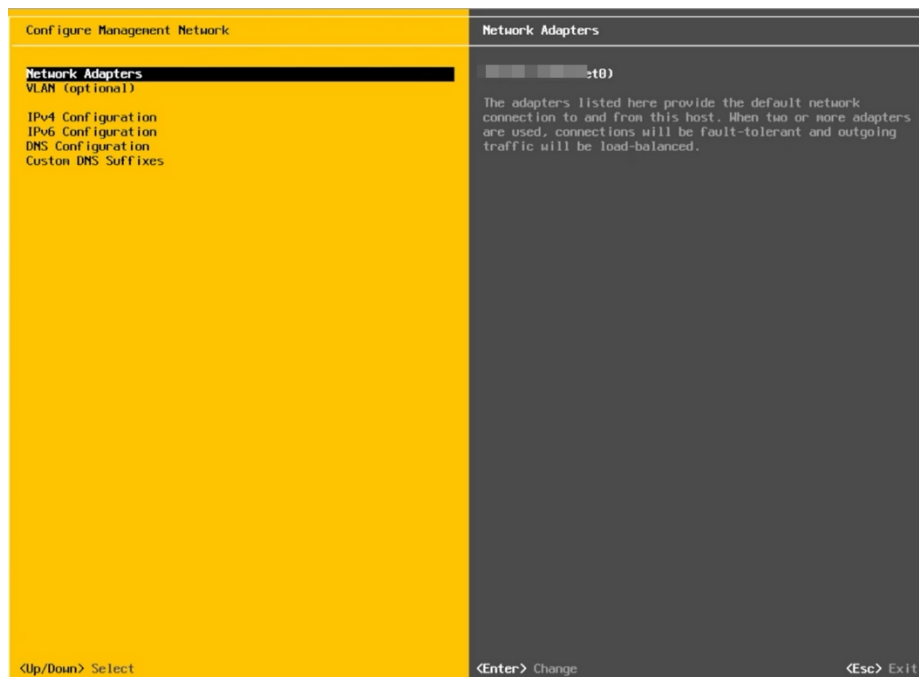
System Customization	Configure Management Network
Configure Password	Hostname: localhost
Configure Lockdown Mode	IPv4 Address: 192. . .
Configure Management Network	Network identity acquired from DHCP server 192. . .
Restart Management Network	IPv6 Addresses: fe80: . . .
Test Management Network	To view or modify this host's management network settings in detail, press <Enter>.
Network Restore Options	
Configure Keyboard	
Troubleshooting Options	
View System Logs	
View Support Information	
Reset System Configuration	
	<Enter> More <Esc> Log Out

3. Set:

- IP address (e.g., 192.168.1.100)
- Subnet mask (255.255.255.0)
- Gateway (192.168.1.1)
- DNS server (8.8.8.8)

Configure Management Network	DNS Configuration
Network Adapters	Automatic
VLAN (optional)	Primary DNS Server: 192. . .
IPv4 Configuration	Alternate DNS Server: Not set
IPv6 Configuration	Hostname: localhost
DNS Configuration	If this host is configured using DHCP, DNS server addresses and other DNS parameters can be obtained automatically. If not, ask your network administrator for the appropriate settings.
Custom DNS Suffixes	
	<Enter> Change <Esc> Exit

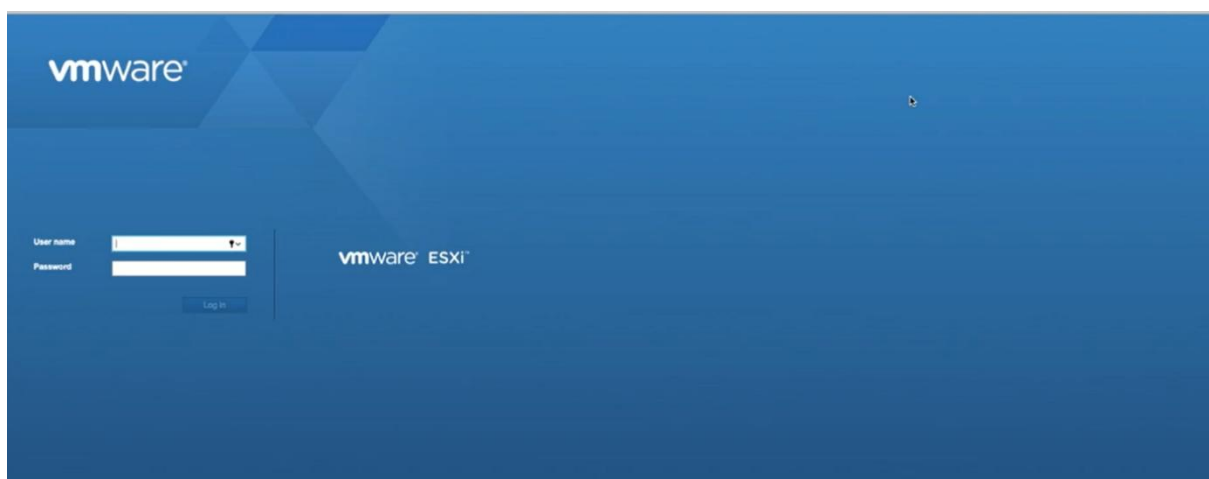
4. Save and restart the management network.



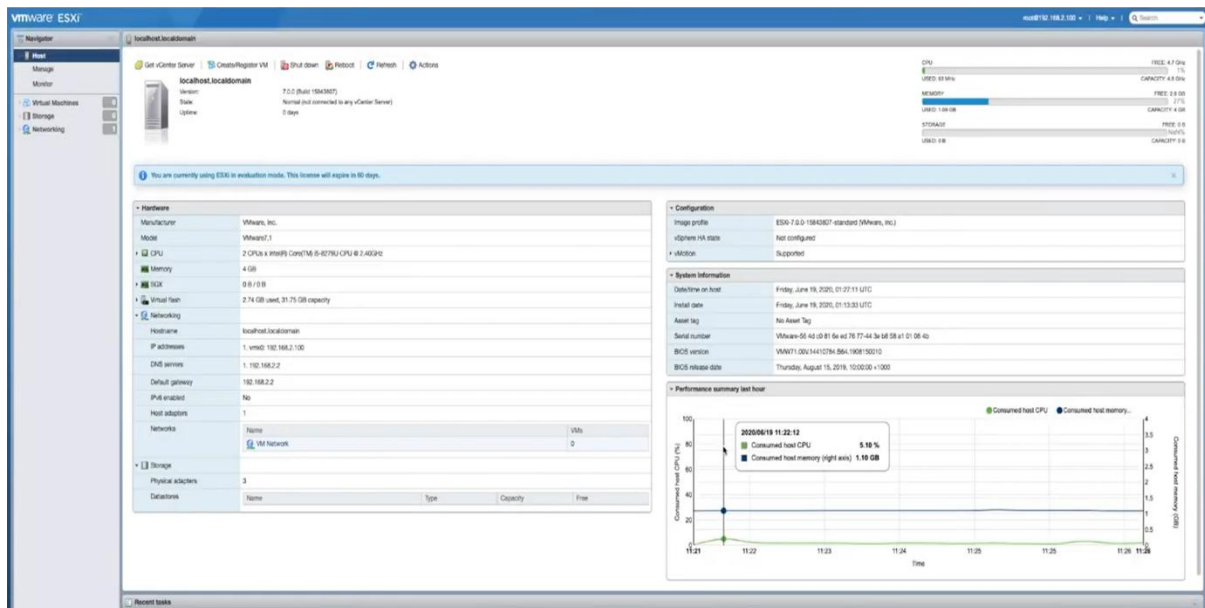
Step 6: Log in to the ESXi Web Client

After installing ESXi, you can log in to the ESXi web client. Make sure the PC is on the same network, and open a browser.

1. Go to <https://<ESXi-IP>> (e.g., <https://192.168.1.100>).
2. Accept the security warning.
3. Log in as **root** with your password.



After login, you will see the following screen:



Step 7: Create a Datastore

1. Go to **Storage → Datastores**.
2. Click **New datastore**.
3. Choose **VMFS** (or NFS/vSAN if needed).
4. Select the disk and give it a name (e.g., datastore1).
5. Decide between Thin provisioning, which uses storage as needed, or Thick provisioning, which reserves the full disk space upfront.
6. Save and confirm.

Step 8: Configure Networking

First, verify that the management network is correctly set up. By default, ESXi creates a management interface named **vmk0**. Make sure this interface exists and is assigned the correct management IP address you configured earlier.

Next, configure a network for your virtual machines. To do this, navigate to **Networking** in the web client, open the **Port Groups** section, and select **Add port group**. Enter a descriptive name such as **VM-Network**, choose an existing vSwitch, and, if your environment uses VLANs, specify the VLAN ID at this stage.

To improve both stability and speed, set up NIC teaming. This involves attaching multiple physical network adapters (NICs) to the same vSwitch so that if one NIC fails, network traffic continues through the other. This setup also helps with load balancing in larger environments.

Step 9: Create Your First Virtual Machine (Example: Ubuntu Server)

1. Navigate to **Virtual Machines** and then go to the **Create / Register VM**.
2. Choose **Create a new virtual machine**.
3. Enter a name (e.g., Ubuntu-Server).
4. Guest OS: **Linux → Ubuntu Linux (64-bit)**.
5. Select your datastore.
6. Assign resources: 2 vCPU, 8 GB RAM, 100 GB disk.
7. Attach your VM to the VM-Network port group.
8. Mount the Ubuntu ISO from the datastore.
9. Finish and power on the VM.
10. Install Ubuntu following the prompts.

Make sure you should check the post-installation checklist:

- Set **NTP** for accurate time.
- Backup host configuration.
- Apply the latest ESXi patches.
- Monitor hardware health status in **Monitor → Hardware**.

Conclusion

You now have a fully functional ESXi installation with storage, networking, and your first VM up and running. From here, you can create more VMs, connect to shared storage, and explore advanced VMware features.

Reference: [*Install VMware ESXi & Create Your First VM: Step-by-Step*](#)

Mini-Project Implementation: Cloud Controller and HDFS Logic

To complete the SaaS requirement over the LAN, a custom Cloud Controller must be implemented using open-source technologies to interface with HDFS (Hadoop Distributed File System).

1. File Segmentation Logic: The controller must include code to divide large files into fixed-size segments (blocks). This allows for distributed storage and parallel processing.

- *Logic:* Read file input stream -> Buffer data into 64MB or 128MB chunks -> Assign block IDs.

2. Encrypted Upload/Download Workflow: To ensure data security within the private cloud, the following logic is implemented:

- **Upload:** File Segment -> AES-256 Encryption (Client/Controller Side) -> Upload to HDFS.
- **Download:** Fetch Block from HDFS -> Decryption using Private Key -> Reassemble Segments into Original File.

For detailed explanation REFERENCE: [Cloud Controller Mini Project for HDFS | PDF | Apache Hadoop | Cloud Computing](#)

Conclusion

The successful setup of a vSphere Private Cloud, combined with a custom HDFS-integrated controller, provides a secure, high-availability platform for delivering Software as a Service (SaaS) over a local network.